

COMPUTER ARCHITECTURE



Week 3: Instruction Set Architecture

Agenda

1

Introduction

ISA definition, CISC vs RISC, design roadmap

2

CPU Internal Storage

Stack, Accumulator, GPR

3

Memory Addressing Modes

Address space, Endianness, Addressing modes

4

Operations in the ISA

Arithmetic, Data transfer, Flow control

5

Instruction Encoding

R / I / S / B / U / J formats, Worked example

6

Compiler Support

How the ISA enables effective compilation

01

Introduction

Why ISA matters · ISA definition · Basic concepts · Design roadmap

Why ISA Matters

The contract between SW & HW

Why does source code run across CPUs after recompilation - but binaries don't?

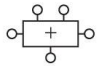
- > It is the language that software speaks and hardware understands
- > Applications target the ISA, not the HW implementation → *portability*
- > Stable ISA → innovate HW while preserving *backward compatibility*

ISA shapes the performance ceiling

The ISA directly affects IC in the Iron law of performance - a key driver of the famous “ISA wars” of the 1980s, with lasting relevance today

	CISC	RISC
Instruction count	Fewer	More
Single instruction	Performs complex operations	Performs simple operations
Program size	Smaller	Larger
HW implementation	Complex decoder	Easier to optimize
Examples	Intel x86, AMD, Cyrix	RISC-V, ARM, SPARC

Instruction Set Architecture (ISA)

Application Software		Programs
Operating Systems		Device Drivers
Architecture		Instructions Registers
Micro-architecture		Datapaths Controllers
Logic		Adders Memories
Digital Circuits		AND Gates NOT Gates
Analog Circuits		Amplifiers Filters
Devices		Transistors Diodes
Physics		Electrons

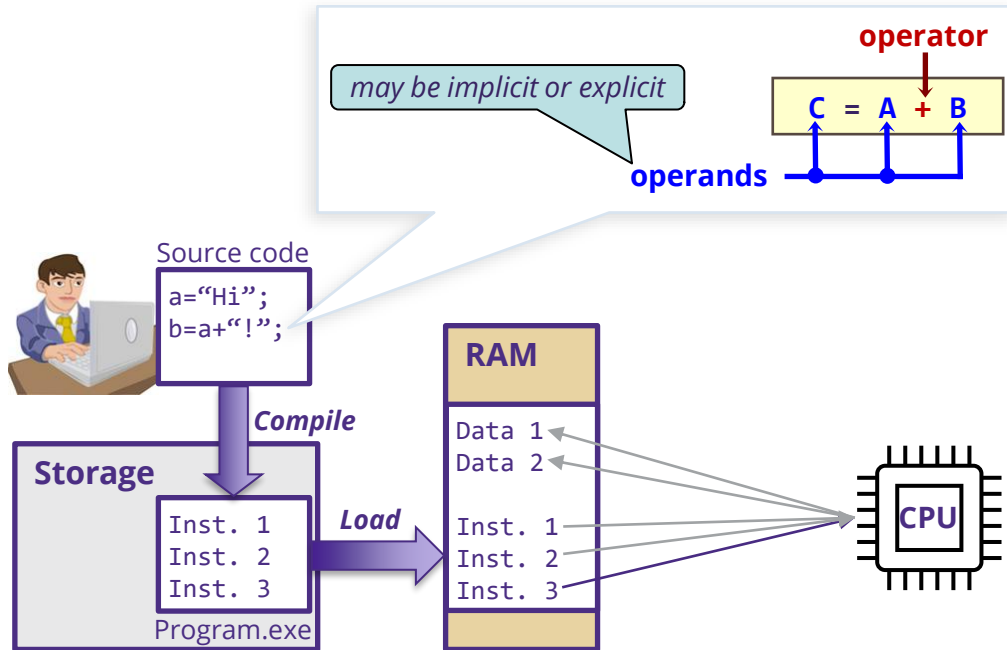


An abstraction that hides HW complexity from programmers, allowing functions to be designed independently from the HW that performs them

Levels of ISA Implementation

- > **Organization** (microarchitecture): how the ISA is implemented (logical blueprint)
Same ISA, different designs (e.g., Intel vs. AMD)
- > **Realization**: physical implementation (circuit/technology)
Same ISA + organization, different chips (e.g., Apple A9)
- > **Key point**: Different ISAs → completely different processors (e.g., x86 vs. ARM)

Programs and Instructions



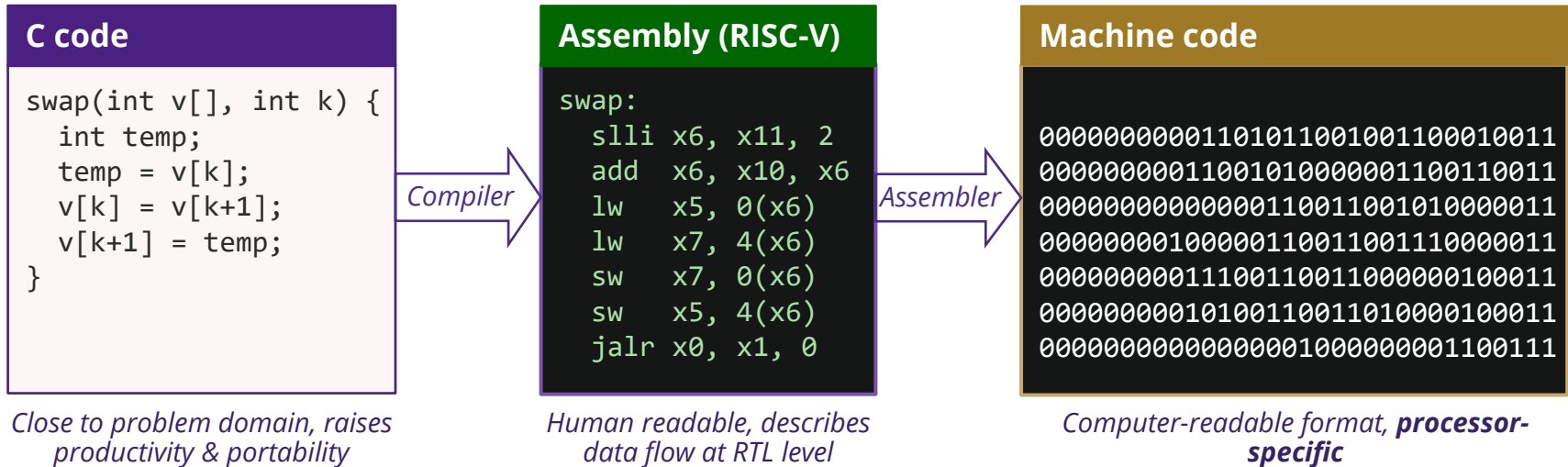
Program

- > A set of instructions that directs a computer to solve a problem
- > Written in HLL and must be converted to machine code for execution

Instruction

- > Fundamental operations that CPU can execute
- > Human analogy: *verbs (operations) applied to objects (operands)*
- > Instruction set: *full vocabulary of a CPU*

From Source Code to Machine Instructions



RISC-V assembly instructions use the format [Label:] Mnemonic [Operands] #Comment, where the mnemonic specifies the operation. Labels/comments are optional, serving as symbolic references & explanations to improve code readability.

Aspects of ISA Design



1

CPU internal storage

How does the CPU store operands when fetching instructions?

2

Addressing modes

How does the CPU interpret & specify addresses in memory?

3

Operations

Which instructions should the ISA support?

4

Encoding

How are instructions encoded into binary bit streams?

5

System SW support

How does the ISA help compilers and OS maximize performance?

Quantitative design methodology

Benchmark candidate ISA configurations and pick the best trade-off.

02

CPU Internal Storage

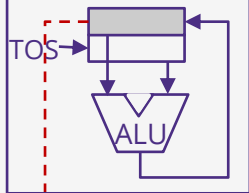
Stack · Accumulator · GPR load-store (RISC-V)

Types of CPU Internal Storage

Stack

```
Push A
Push B
Add
Pop C
```

Processor



Memory

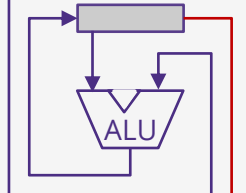


Operands implicitly on TOS

Accumulator

```
Load A
Add B
Store C
```

Processor



Memory

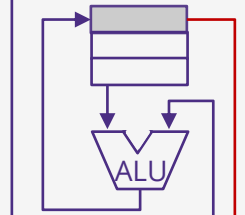


ACC is implicit operand

Reg-Memory

```
Load R1, A
Add R3, R1, B
Store R3, C
```

Processor



Memory

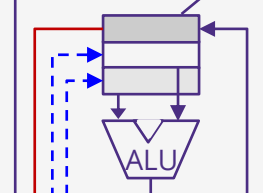


One mem operand allowed

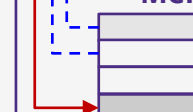
Reg-Reg (RISC-V)

```
Load R1, A
Load R2, B
Add R3, R1, R2
Store R3, C
```

Processor



Memory



Registers
*fast memory
built directly
inside the CPU*

All ops use registers only

RISC-V Design Principles Applied to GPRs



1 Simplicity favors regularity

Uniform GPR width fixed-length 32-bit instructions; consistent field positions across formats.

2 Make the common case fast

32/64-bit registers match the width of the most frequent data types. Simple operations are cheap.

3 Smaller is faster

32 GPRs — enough for most programs, few enough for a fast register file with short access time.

4 Good design demands good compromises

32 registers balances speed vs flexibility. Fixed encoding balances code density vs decoder simplicity.

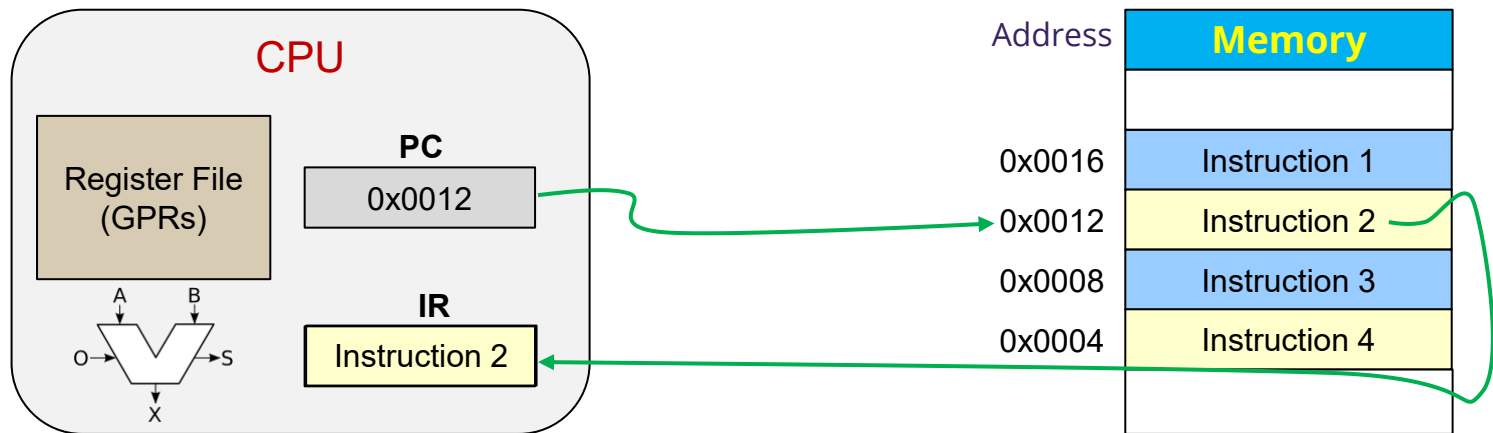
RISC-V General Registers

ABI Name / Register	Usage
zero / x0	Constant value 0 — always reads as 0, writes ignored
ra / x1	Return address — stores PC+4 on function call
sp / x2	Stack pointer — points to top of current stack frame
gp / x3	Global pointer — base for global data access
tp / x4	Thread pointer — per-thread data
t0-t2 / x5-7	Temporaries — caller-saved; not preserved across calls
s0/fp / x8	Saved register / Frame pointer — callee-saved
s1 / x9	Saved register — callee-saved
a0-a1 / x10-11	Function arguments AND return values
a2-a7 / x12-17	Additional function arguments
s2-s11 / x18-27	Saved registers — callee-saved
t3-t6 / x28-31	Temporaries — caller-saved

Special Registers

> There are other registers in RISC-V architecture!

- Not visible to user
- E.g., **PC** (Program Counter) is used to hold the address of the next instruction to be fetched from memory.
- E.g., **IR** (Instruction Register) is used to hold the binary code of the instruction which PC points to.



03

Memory Addressing Modes

Address space · Endianness · RV32I addressing

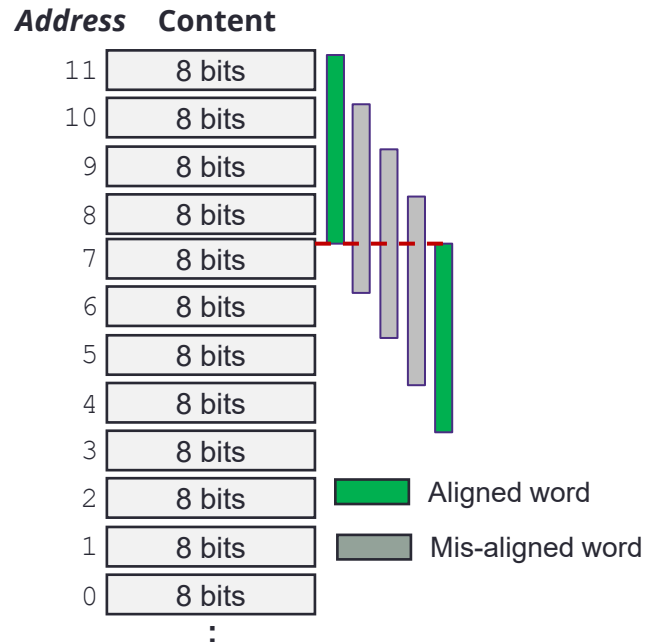
Addressing Modes

Addressing modes: ways to obtain an operand of an instruction

<u>Addressing mode</u>	<u>Example</u>	<u>Meaning</u>	
Register	Add R4, R3	$R4 \leftarrow R4 + R3$	RISC-V uses only the top 3 modes ▪ <i>Smaller is faster</i> ▪ <i>Good design demands good compromises</i>
Immediate	Add R4, #3	$R4 \leftarrow R4 + 3$	
Displacement	Add R4, 100(R1)	$R4 \leftarrow R4 + \text{Mem}[100 + R1]$	
Register indirect	Add R4, (R1)	$R4 \leftarrow R4 + \text{Mem}[R1]$	
Indexed / Base	Add R3, (R1+R2)	$R3 \leftarrow R3 + \text{Mem}[R1 + R2]$	
Direct or absolute	Add R1, (1001)	$R1 \leftarrow R1 + \text{Mem}[1001]$	
Memory indirect	Add R1, @(R3)	$R1 \leftarrow R1 + \text{Mem}[\text{Mem}[R3]]$	
Auto-increment	Add R1, (R2)+	$R1 \leftarrow R1 + \text{Mem}[R2]; R2 \leftarrow R2 + d$	
Auto-decrement	Add R1, -(R2)	$R2 \leftarrow R2 - d; R1 \leftarrow R1 + \text{Mem}[R2]$	
Scaled	Add R1, 100(R2)[R3]	$R1 \leftarrow R1 + \text{Mem}[100 + R2 + R3 * d]$	

Memory Address Space

- > Memory is a large 1D array; each location has an address
 - k-bit address $\rightarrow$ address space of 2^k (RV32I: 32-bit $\rightarrow$ 4 GB)
 - 1 byte per address $\rightarrow$ byte-addressable
- > A word = 4 bytes; word address = address of its first byte
 - Consecutive word addresses are 4 bytes apart
 - Aligned word: begins at a byte address that is a multiple of 4

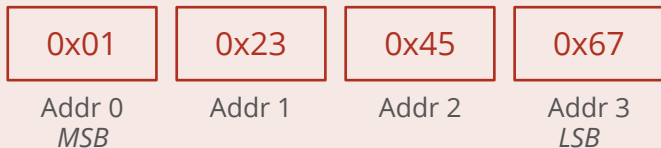


Endianness

Endianness = the relative ordering of bytes within a multi-byte word in memory.

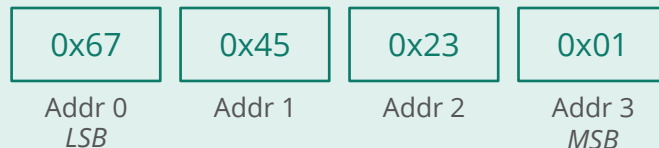
Big-endian (e.g. SPARC, some network protocols)

MSB stored at lowest address



Little-endian (RISC-V, x86)

LSB stored at lowest address



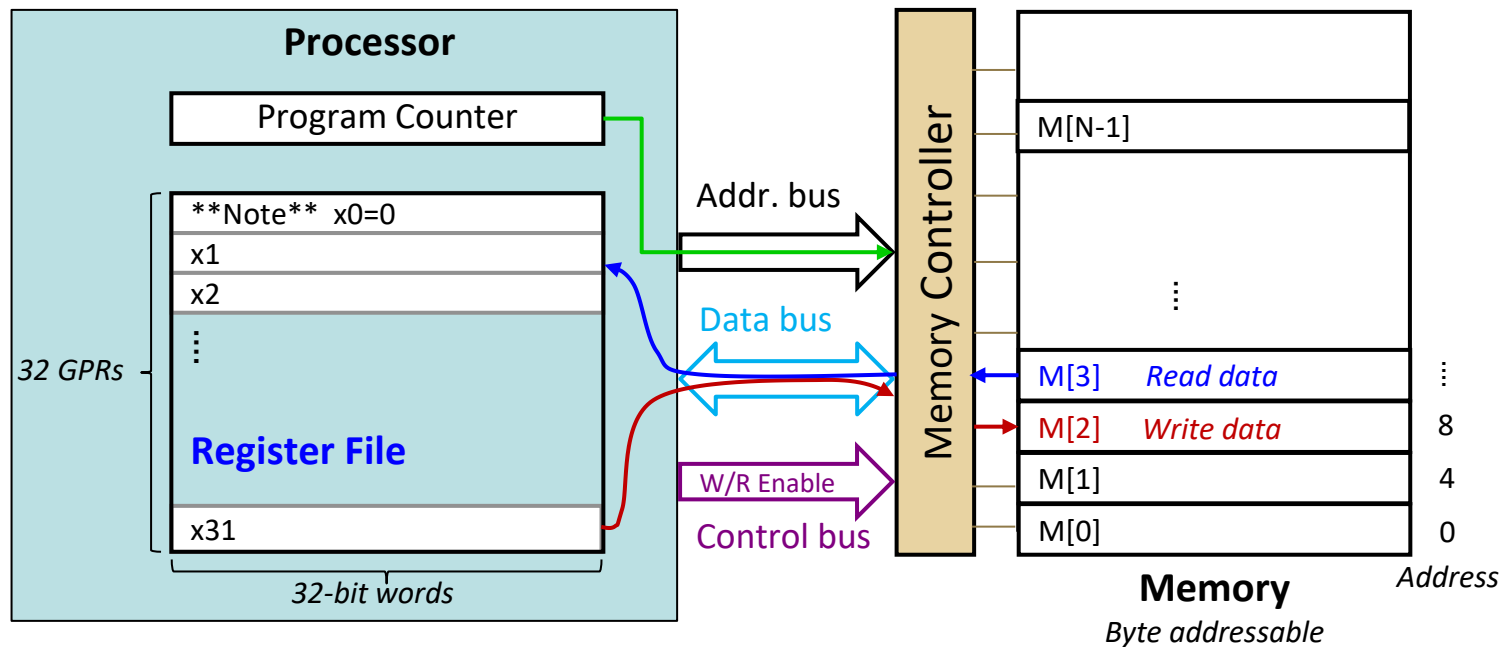
Worked example: Show how computers store the 64-bit value **0x0123456789ABCDEF**, starting at address 0.

Big-endian: EF CD AB 89 67 45 23 01
Address: 0 1 2 3 4 5 6 7

Little-endian: 01 23 45 67 89 AB CD EF
Address: 0 1 2 3 4 5 6 7

💡 *Endianness defines byte order within a multi-byte object – not the order of separate words or the bit order within a byte.*

RV32I Memory Access Mechanism



Example

C source

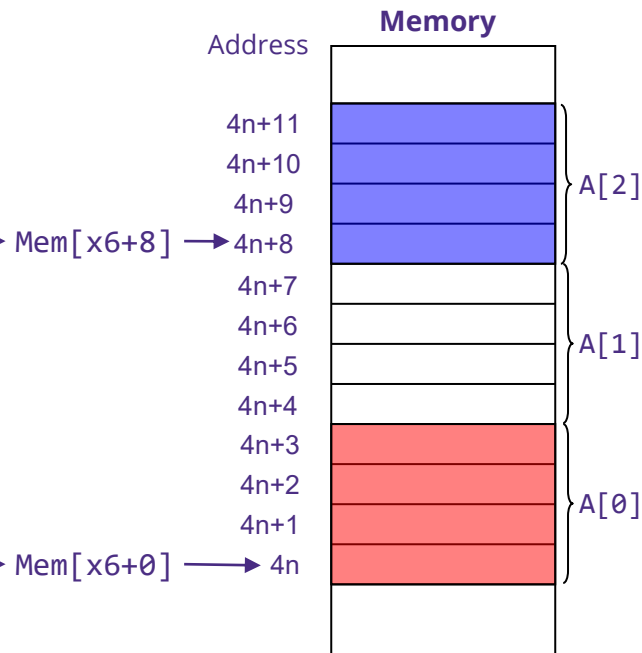
```
A[0] = h + A[2];
```

Translate to RISC-V assembly

x6 = base address of A[] (i.e., &A[0])
x7 = h (value of variable h)

RISC-V Assembly

```
lw    x5, 8(x6)  → displacement mode: base = x6, offset = 8  
add   x5, x5, x7  → register mode: x5 ← x5 + x7  
sw    x5, 0(x6)  → displacement mode: base = x6, offset = 0
```



04

Operations in the ISA

Arithmetic · Data transfer · Flow control

Supported Operations

Operator type	Examples
Arithmetic and logical	Integer arithmetic and logical operations: add, subtract, and, or, multiply, divide
Data transfer	Loads-stores (move instructions on computers with memory addressing)
Control	Branch, jump, procedure call and return, traps
System	Operating system call, virtual memory management instructions
Floating point	Floating-point operations: add, multiply, divide, compare
Decimal	Decimal add, decimal multiply, decimal-to-character conversions
String	String move, string compare, string search
Graphics	Pixel and vertex operations, compression/decompression operations

We focus only on this subset of operators to illustrate the RV32I ISA

- *Smaller is faster*
- *Make the common case fast*

RV32I: Arithmetic & Data Transfer

Arithmetic & Logical

```
add  rd, rs1, rs2  # rd ← rs1 + rs2
addi rd, rs1, imm  # rd ← rs1 + imm
sub  rd, rs1, rs2  # rd ← rs1 - rs2
sll  rd, rs1, rs2  # rd ← rs1 << rs2 (logical)
slli rd, rs1, imm  # rd ← rs1 << imm (logical)
sra  rd, rs1, rs2  # rd ← rs1 >> rs2 (arithmetic)
and  rd, rs1, rs2  # bitwise AND
ori  rd, rs1, imm  # bitwise OR
lui  rd, imm       # rd ← imm << 12
```

Data Transfer (Load / Store)

```
lw  rd, imm(rs1)  # rd ← Mem[rs1+imm] (32-bit)
lh  rd, imm(rs1)  # rd ← Mem[rs1+imm][0:15]
lb  rd, imm(rs1)  # rd ← Mem[rs1+imm][0:7]
lbu rd, imm(rs1)  # unsigned byte load
sw  rs2, imm(rs1) # Mem[rs1+imm] ← rs2 (32-bit)
sh  rs2, imm(rs1) # Mem[rs1+imm][0:15] ← rs2
sb  rs2, imm(rs1) # Mem[rs1+imm][0:7] ← rs2
```

Design note: RISC-V has no `subi` instruction (Design Principle 1) as `addi rd, rs1, -imm` can be utilized. Likewise, there is no `not` or `nor` as they can be synthesized, e.g., `not x ≡ xori x, x, -1`

RV32I: Flow Control Instructions

Branch instructions

```
beq  rs1, rs2, Label    # branch if rs1 == rs2
bne  rs1, rs2, Label    # branch if rs1 != rs2
blt   rs1, rs2, Label    # branch if rs1 < rs2
bge   rs1, rs2, Label    # branch if rs1 >= rs2
bltu  rs1, rs2, Label    # unsigned <
bgeu  rs1, rs2, Label    # unsigned >=
```

Why no ble/bgt? Design principle 1: swap operands to achieve the same effect: `bgt a, b` $\equiv$ `blt b, a`

Jump instructions

```
jal  rd, offset        # rd<PC+4; PC<PC+offset
j    offset             # pseudo: jal x0, offset
jalr rd, rs1, imm       # rd<PC+4;
                               # PC<(rs1+imm)&~1
auipc rd, imm           # rd<PC+(imm<<12)
```

Note: `jalr` masks the LSB to 0 so that PC is always *half-word* aligned, per the RISC-V specification.

05

Instruction Encoding

R · I · S · B · U · J formats · Worked example

RV32I Encoding Design Choices

Encoding: how instructions are represented in binary format for execution by the processor.

Things to be considered

- > Number of registers
- > Number of operands in an instruction
- > Number of addressing modes
- > Instruction length

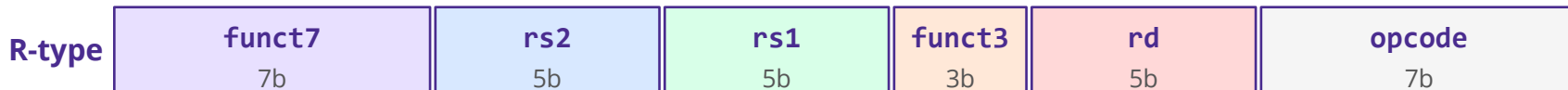
Competing forces

- > Register count vs. Operand space
- > Number of operands vs. Code size
- > Rich addressing modes vs. Encoding complexity
- > Instruction length vs. Implementation difficulty

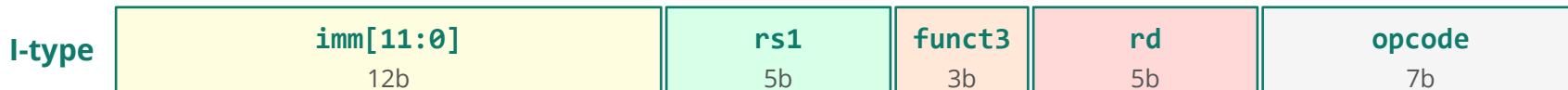
Example: Code size (3- vs. 2-operand instructions)

- > 3-operand: `add x1, x2, x3` → 1 instruction (32 bits)
- > 2-operand, `:mov x1, x2; add x1, x3` → 2 instructions (2x16 = 32 bits)
- > Operations are often *in-place* (e.g., `i = i + 1`) → `add x1, x2` (16 bits). Fewer operands → better code size.

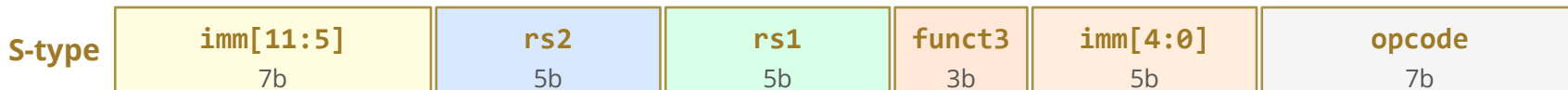
RV32I Instruction Formats: R, I, S



Used for: add, sub, and, or, xor, sll, srl, sra ...



Used for: addi, lw, lh, lb, jalr ... Same field positions as R-type where possible → simpler HW



Used for: sw, sh, sb ... Immediate split to keep rs1 & rs2 in same positions as R/I-type - simplifies register-file read logic (*Simplicity favors regularity*)

RV32I Instruction Formats: B, U, J

B-type

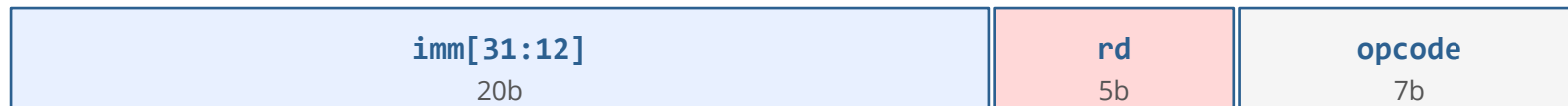


Used for: beq, bne, blt, bge ... Immediate layout split to maximize bit reuse with S-type.

imm: signed branch offset (**in half-words**) relative to the current PC → imm[0] = 0 (not encoded).

imm[11] placed next to imm[4:1] to share sign-extension HW across formats (I/S/U/J) - reduces HW cost.

U-type



Used for: lui, auipc. Loads a 20-bit immediate into the upper 20 bits of rd (bits 31:12).

J-type



Used for: jal. Target = PC + sign_extend({imm[20:1], 1'b0}) i.e. offset is always even (half-word aligned).

Example

Encoding add x5, x6, x7 in binary (R-type)

opcode	7 bits	011 0011	Integer register-register ops (R-type)
rd	5 bits	00 101	x5 = register 5 = binary 00101
funct3	3 bits	000	Specifies ADD (vs SLT etc.)
rs1	5 bits	00 110	x6 = register 6 = binary 00110
rs2	5 bits	00 111	x7 = register 7 = binary 00111
funct7	7 bits	000 0000	ADD (funct7=0000000 vs SUB=0100000)

Assembled: 0000000 00111 00110 000 00101 0110011 = 0x00730233

06

Support for Compiler

How RV32I enables effective compilation

How RISC-V Supports Compilers

The ISA must be designed so compilers can generate efficient, correct code.

x0 hardwired to zero

- > Simplifies code generation enormously, for example, `mov rd, 0` becomes `addi rd, x0, 0`

32 uniform GPRs

- > Enough registers for the allocator to avoid spills in most loops
- > Uniformity means no special-case handling

Fixed 32-bit encoding

- > All instructions are the same width
- > The compiler can predict instruction addresses statically

No condition codes

- > Each branch reads registers directly, no hidden state
- > Makes compiler branch scheduling simpler

KEY TAKEAWAYS

1

ISA is the HW/SW contract

The ISA defines what SW can ask HW to execute - independently of the μ -architecture implementation

2

Four RISC-V design principles

Regularity, Common Case Fast, Smaller is Faster, Good Compromises

3

Load-store architecture

All computation happens in registers. Only lw/sw touch memory. More IC but consistent CPIs, simpler HW.

4

Six fixed-width encoding formats

R/I/S/B/U/J — all 32 bits wide. Field positions are reused across formats to minimize decoder hardware.

5

RISC: do less, do it simply

No subi, no not, no ble. Compilers synthesize complex operations from simple primitives.

Next Lecture: Programming with RV32I — P&H sections 2.7, 2.8, 2.10, 2.12, 2.14